# Firebox Cyber Toolkit

## Complete Command Guide

Commands, copyable examples, usage, and safety limits

---

Production CLI reference

Version 0.1.0 • Generated 28 October 2026

# Contents


1 Read this first ..... 1
1.1 Install and update ..... 1
1.2 Global commands ..... 1
1.2.1 Help ..... 1
1.2.2 Version ..... 2
1.3 Short operator aliases ..... 2
2 Network and DNS ..... 2
2.1 TCP scan ..... 2
2.2 Specific TCP ports ..... 3
2.3 DNS lookup ..... 3
3 Web security commands ..... 3
3.1 Passive web check ..... 3
3.2 Bounded passive web audit ..... 3
3.3 Authorized read-only path enumeration ..... 4
4 API documentation discovery ..... 4
5 Authentication log analysis ..... 5
6 Password, token, and hash commands ..... 5
6.1 Secure password generation ..... 5
6.2 Password strength ..... 5
6.3 Secure token generation ..... 6
6.4 SHA-256 hash generation ..... 6
6.5 SHA-256 verification ..... 6
7 File integrity monitoring ..... 6
7.1 Create a baseline ..... 6
7.2 Compare against a baseline ..... 6
8 Metadata and system commands ..... 7
8.1 File metadata ..... 7
8.2 System information ..... 7
9 Wi-Fi and PCAP commands ..... 7
9.1 Wireless interface status ..... 7
9.2 Read-only interface audit ..... 7
9.3 PCAP metadata ..... 7
10 Phishing-awareness commands ..... 8
10.1 Analyze an email file ..... 8
10.2 Generate a synthetic training template ..... 8
11 Vulnerability records ..... 8
11.1 Add a finding ..... 8
11.2 List findings ..... 9
12 Safe defensive replacement tools ..... 9
12.1 Phone-number format validation ..... 9

12.2 Passive CMS and technology inspection ..... 9
12.3 PDF information ..... 10
12.4 Optional local OCR ..... 10
12.5 Local process inventory ..... 10
12.6 Read-only VPN configuration check ..... 10
13 Manual Python plugins ..... 10
14 Output, reports, and colors ..... 11
15 Production safety checklist ..... 11

# 1 Read this first

Firebox is a lightweight defensive security toolkit for systems and websites that you own or are explicitly authorized to assess. The normal output is readable terminal text. Add `--json` when another program needs machine-readable output, and add `--report PATH` when a JSON report file is required.

The toolkit does not perform credential theft, password cracking, Wi-Fi key attacks, phishing delivery, exploit payloads, destructive requests, unrestricted crawling, or state-changing API calls. The bounded web enumerator is path enumeration only and requires an explicit authorization flag.

## 1.1 Install and update

```
# Install on Ubuntu 24.04
sudo apt-get update && sudo apt-get install -y git python3 python3-venv
git clone https://github.com/njogu26713-commits/firebox-cyber-toolkit.git
cd firebox-cyber-toolkit
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e .
python3 -m unittest discover -s tests -v
firebox --help
```

If the environment already exists:

```
cd ~/firebox-cyber-toolkit
git pull --ff-only
source .venv/bin/activate
python3 -m pip install -e .
hash -r
```

If `.venv/bin/activate` does not exist:

```
sudo apt-get update
sudo apt-get install -y python3-venv
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e .
```

## 1.2 Global commands

### 1.2.1 Help

Show all top-level modules and options:

```
firebox --help
```

Show help for a module or command:

```
firebox network --help
firebox web audit --help
firebox web enumerate --help
```

### 1.2.2 Version

```
firebox --version
```

## 1.3 Short operator aliases

These aliases are designed for quick, readable terminal use. The longer forms remain available.

| Short command | Equivalent operation |
|---|---|
| `firebox scan HOST` | `network scan --target HOST` |
| `firebox ports HOST --ports 22,443` | `network ports --target HOST --ports 22,443` |
| `firebox dns DOMAIN` | `dns lookup --domain DOMAIN` |
| `firebox web URL` | `web check --url URL` |
| `firebox vuln-scan URL` | `web audit --url URL` |
| `firebox logs FILE` | `logs analyze --file FILE` |
| `firebox hash FILE` | `hash generate --file FILE` |

```
firebox scan 127.0.0.1
firebox dns example.com
firebox web https://your-authorized-domain.example
firebox vuln-scan https://your-authorized-domain.example
```

# 2 Network and DNS

## 2.1 TCP scan

`network scan` checks TCP connectivity against an explicitly supplied host. Without `--ports`, it checks ports 1–1024. It uses bounded concurrency and a timeout. Closed ports are summarized in normal output; JSON and reports preserve the detailed result.

```
# Localhost, default bounded range 1-1024
firebox scan 127.0.0.1

# Explicit authorized scope
firebox scan chukanest.co.ke --ports 22,80,443

# Tune timeout and concurrency
firebox scan 127.0.0.1 --ports 22,80,443 --timeout 2 --workers 4
```

```
# Save a complete JSON report
firebox scan 127.0.0.1 --report reports/localhost-scan.json

# Machine-readable output
firebox scan 127.0.0.1 --json
```

Long form:

```
firebox network scan --target 127.0.0.1 --ports 22,80,443
```

## 2.2 Specific TCP ports

Use `network ports` when you want to check only a small list sequentially.

```
firebox ports 127.0.0.1 --ports 22,80,443
firebox network ports --target chukanest.co.ke --ports 80,443 --timeout 3
```

## 2.3 DNS lookup

```
firebox dns chukanest.co.ke
firebox dns chukanest.co.ke --json
firebox dns lookup --domain chukanest.co.ke --report reports/dns.json
```

DNS resolution uses the server's system resolver. It does not claim ownership or geolocation.

# 3 Web security commands

## 3.1 Passive web check

`web check` sends a safe `HEAD` request and reports the response status, headers, cookies, redirects, TLS validation, and certificate expiry. It does not submit form data or exploit anything.

```
firebox web https://chukanest.co.ke
firebox web https://chukanest.co.ke --report reports/chukanest-web-check.json
firebox web check --url https://chukanest.co.ke --json
```

## 3.2 Bounded passive web audit

`web audit` checks one base URL or up to eight explicitly supplied paths. Findings are evidence-based from live responses. It is not a complete penetration test.

```
firebox vuln-scan https://chukanest.co.ke

firebox web audit \
  --url https://chukanest.co.ke \
```

```
  --paths /,/login,/health,/api,/openapi.json \
  --report reports/chukanest-web-audit.json
```

It checks TLS, certificate expiry, HTTP status, server errors, HSTS, CSP, X-Content-Type-Options, Referrer-Policy, cookie attributes, and redirect information.

## 3.3 Authorized read-only path enumeration

This is the only command in this guide that accepts a wordlist. It requires `--confirm-authorized`, a user-supplied wordlist, sequential requests, a maximum of 500 requests, and a minimum delay of 0.1 seconds. It uses `HEAD` only. It is path enumeration, not password or credential brute force.
Create a reviewed wordlist:

```
cat > approved-paths.txt <<'EOF'
/
/login
/health
/api
/openapi.json
/swagger.json
EOF
```

Run it:

```
firebox web enumerate \
  --url https://chukanest.co.ke \
  --wordlist approved-paths.txt \
  --confirm-authorized \
  --max-requests 100 \
  --delay 0.5
```

Save a report:

```
firebox web enumerate \
  --url https://chukanest.co.ke \
  --wordlist approved-paths.txt \
  --confirm-authorized \
  --max-requests 100 \
  --delay 0.5 \
  --report reports/path-enumeration.json
```

# 4 API documentation discovery

`api discover` checks standard OpenAPI and Swagger documentation paths, or up to eight paths you explicitly provide. It performs bounded read-only `GET` requests. It does not crawl, authenticate, brute-force endpoints, or call state-changing methods.

```
firebox api discover --url https://chukanest.co.ke

firebox api discover \
  --url https://chukanest.co.ke \
  --paths /openapi.json,/swagger.json,/api/docs \
  --report reports/chukanest-api-discovery.json

firebox api discover --url https://chukanest.co.ke --json
```

# 5 Authentication log analysis

Analyze a supplied Linux authentication log for repeated failures, targeted usernames, suspicious sources, and successful logins following failures.

```
# Included synthetic fixture
firebox logs samples/sample-auth.log

# Custom threshold
firebox logs samples/sample-auth.log --threshold 3

# Authorized system log, if readable by the current user
firebox logs /var/log/auth.log --report reports/auth-analysis.json
```

No root access is required for a readable supplied file. Do not weaken permissions on protected logs.

# 6 Password, token, and hash commands

## 6.1 Secure password generation

Generated secrets are printed to stdout only. They are not written to application logs.

```
firebox password generate --length 20
firebox password generate --length 32
```

Supported length: 12–256 characters.

## 6.2 Password strength

Use test values rather than real production passwords in shell history.

```
firebox password strength --password 'Example-Test-Password-2026!'
firebox password strength --password 'Use-a-test-value-only'
```

## 6.3 Secure token generation

```
firebox password token --length 32
firebox password token --length 64
```

Supported length: 16–256 bytes.

## 6.4 SHA-256 hash generation

```
firebox hash README.md
firebox hash generate --file README.md
firebox hash README.md --report reports/readme-hash.json
```

## 6.5 SHA-256 verification

```
firebox hash verify \
  --file README.md \
  --expected YOUR_EXPECTED_SHA256_HASH

firebox hash verify \
  --file README.md \
  --expected "$(sha256sum README.md | awk '{print $1}')"
```

# 7 File integrity monitoring

## 7.1 Create a baseline

Baselines are not overwritten unless `--force` is provided.

```
firebox integrity baseline \
  --path samples \
  --baseline data/samples-baseline.json

firebox integrity baseline \
  --path /path/to/monitored-directory \
  --baseline /secure/path/firebox-baseline.json
```

## 7.2 Compare against a baseline

```
firebox integrity scan \
  --path samples \
  --baseline data/samples-baseline.json

firebox integrity scan \
```

```
  --path /path/to/monitored-directory \
  --baseline /secure/path/firebox-baseline.json \
  --json
```

Reports identify modified, added, missing, and unreadable-file conditions where applicable.

# 8 Metadata and system commands

## 8.1 File metadata

```
firebox metadata inspect --file README.md
firebox metadata inspect --file /path/to/authorized/document.pdf
firebox metadata inspect --file /path/to/image.jpg --json
```

The current metadata command reports existence, size, a type hint, and SHA-256. It does not claim to extract all PDF or image EXIF fields.

## 8.2 System information

```
firebox system info
firebox system info --json
```

# 9 Wi-Fi and PCAP commands

These are local defensive diagnostics only. They do not scan nearby networks, crack keys, capture credentials, inject frames, or deauthenticate clients. Azure cloud VMs commonly have no wireless adapter.

## 9.1 Wireless interface status

```
firebox wifi status
firebox wifi status --json
```

If no local wireless interface is present, the result explains that the host may be a wired-only or cloud VM.

## 9.2 Read-only interface audit

Use the actual local wireless interface name, if one exists.

```
firebox wifi audit --interface wlan0
firebox wifi audit --interface wlan0 --json
```

The command reads local interface state and, when available, read-only `iw` link information.

## 9.3 PCAP metadata

Use a real authorized classic PCAP file. The example filename must exist.

```
find ~ -type f \( -name '*.pcap' -o -name '*.pcapng' \) -print
firebox wifi pcap --file /actual/path/to/capture.pcap
firebox wifi pcap --file /actual/path/to/capture.pcap --json
```

The current parser reports classic PCAP metadata and packet record lengths. It does not decode payloads or extract credentials and limits input to 256 MiB.

# 10 Phishing-awareness commands

These commands are defensive. Analysis is static and never opens links. Templates are synthetic and non-deliverable.

## 10.1 Analyze an email file

The repository includes a safe synthetic sample.

```
firebox phishing analyze --file samples/suspicious-email.eml
firebox phishing analyze \
  --file samples/suspicious-email.eml \
  --report reports/phishing-analysis.json
firebox phishing analyze --file suspicious-email.eml --json
```

The analyzer checks From/Reply-To differences, authentication-result headers, urgency language, non-HTTPS links, non-ASCII hostnames, and attachments.

## 10.2 Generate a synthetic training template

```
firebox phishing template --scenario account-alert
firebox phishing template --scenario suspicious-invoice --output reports/training-
template.txt
firebox phishing template --scenario delivery-notice --output reports/delivery-
training.txt
```

Allowed scenarios are `suspicious-invoice`, `account-alert`, and `delivery-notice`. Output contains no real-brand impersonation, live links, tracking, credential collection, or delivery mechanism.

# 11 Vulnerability records

The vulnerability database is local SQLite at `data/firebox.db` by default.

## 11.1 Add a finding

```
firebox vulns add \
  --title 'Missing security header' \
  --severity medium \
  --asset 'https://your-authorized-domain.example' \
```

```
  --evidence 'The authorized passive assessment did not observe CSP' \
  --remediation 'Configure and test an appropriate CSP policy'
```

Allowed severity values: `critical`, `high`, `medium`, `low`, `info`.
Add a status explicitly:

```
firebox vulns add \
  --title 'Outdated package' \
  --severity high \
  --asset 'authorized-server' \
  --evidence 'Installed version is below the approved baseline' \
  --remediation 'Upgrade and verify dependent services' \
  --status in-progress
```

Allowed statuses: `open`, `in-progress`, `resolved`.

### 11.2 List findings

```
firebox vulns list
firebox vulns list --status open
firebox vulns list --status in-progress
firebox vulns list --status resolved
firebox vulns list --json
```

## 12 Safe defensive replacement tools

These commands provide defensive alternatives to tracking, cracking, stealth, exploitation, and data-theft tools. They are intentionally limited to format validation, passive inspection, or local user-supplied files.

### 12.1 Phone-number format validation

This checks formatting only. It does not identify a person, locate a phone, query a carrier, or perform surveillance.

```
firebox phone validate --number '+254700000000'
firebox phone validate --number '+254700000000' --json
```

### 12.2 Passive CMS and technology inspection

This uses observable response headers only. It does not crawl or exploit the site and does not claim positive identification when evidence is inconclusive.

```
firebox cms inspect --url https://your-authorized-domain.example
firebox cms inspect --url https://your-authorized-domain.example --json
```

## 12.3 PDF information

Inspect a user-supplied PDF signature, version, size, and SHA-256 hash.

```
firebox document pdf-info --file report.pdf
firebox document pdf-info --file report.pdf --json
```

## 12.4 Optional local OCR

OCR uses the locally installed Tesseract executable when available. If it is missing, Firebox reports that clearly instead of pretending extraction occurred.

```
firebox document ocr --file image.png
firebox document ocr --file scanned-document.jpg --json
```

## 12.5 Local process inventory

This reports local process names, PIDs, and UIDs. It does not automatically label a process malicious or terminate anything.

```
firebox system processes
firebox system processes --json
```

## 12.6 Read-only VPN configuration check

Review an authorized VPN configuration without modifying it or printing credentials.

```
firebox config vpn-check --file vpn.conf
firebox config vpn-check --file vpn.conf --json
```

# 13 Manual Python plugins

Trusted Python files placed in `plugins/` are loaded automatically at startup. The plugin directory contains the full contract in `plugins/README.md`.

Create a plugin:

```
# plugins/system_check.py

def register(subparsers):
    parser = subparsers.add_parser("system-check", help="Run a custom system
check")
    parser.add_argument("--path", default="/etc/os-release")
    parser.set_defaults(_plugin_handler=run)


def run(args):
    with open(args.path, "r", encoding="utf-8") as file:
```

```
        content = file.read(4096)
    return {"plugin": "system-check", "path": args.path, "readable": True,
"preview": content}
```

Use it:

```
firebox plugins list
firebox system-check --help
firebox system-check --path /etc/hostname
```

Use a separate plugin directory:

```
FIREBOX_PLUGIN_DIR=/secure/path/plugins firebox plugins list
```

Plugins execute with the same operating-system permissions as the current user. Review source before use. A plugin can read files, make network connections, modify data, or execute arbitrary Python code. Built-in commands do not depend on plugins.

# 14 Output, reports, and colors

Readable text is the default. Use JSON for automation:

```
firebox scan 127.0.0.1 --json
firebox web audit --url https://your-authorized-domain.example --json
```

Use `--report PATH` where the command supports it:

```
firebox web audit --url https://your-authorized-domain.example --report reports/
audit.json
firebox api discover --url https://your-authorized-domain.example --report reports/
api.json
```

Colors are enabled in interactive terminals. They are disabled for redirected output, JSON, and `NO_COLOR=1`:

```
NO_COLOR=1 firebox scan 127.0.0.1
```

# 15 Production safety checklist

Before any network or web operation:

- Confirm written authorization and the exact target scope.
- Prefer a staging environment with synthetic data.
- Use narrow paths, small port lists, and conservative timeouts.
- Preserve reports securely; do not commit private logs or scan results.
- Do not expose a dashboard or plugin directory publicly.
- Never put production passwords, tokens, or API keys in command history or reports.
- Treat every manually added plugin as executable code and review it first.

This guide describes the implemented Firebox behavior. It does not claim that passive checks prove exploitability or replace a full authorized penetration test.